



Testing a solver against exact solutions

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, benchmarks, verification

A geodynamics solver is hard to test because the thing it computes is the thing you do not know. Grid refinement tells you a calculation is converging, but not what it is converging to. Comparing two codes tells you they agree, which is worth something and is not correctness. The one check that settles the question is a problem whose answer can be written down.

Underworld3 ships thirteen of those, as `uw.analytic`: the Velic family of manufactured and semi-analytic Stokes solutions that the geodynamics community has used for two decades, plus an elliptical inclusion, a cylindrical annulus flow, and a set of scalar transport solutions. They cover a viscosity jump, an exponentially varying viscosity, a laterally oscillating one, a power-law rheology, a dense block in three dimensions.

This note is about the part that turned out to matter more than the solutions themselves: how you check that an exact solution is exact. We wrote the checks so that they consult no reference — not the paper, not the C kernel the transcription came from, not the solver being tested. Each one is formed from the solution's own symbolic fields by differentiation. That decision found four defects. Two of them are in sources that have been vendored, copied and reused since the 1990s, and one of those is invisible at the parameter value everyone runs.

1. AN EXACT SOLUTION IS ONLY AS GOOD AS THE CHECK YOU APPLY TO IT

There are three ways to test a transcribed solution, and they are not equally strong.

Compare it against the kernel it was transcribed from. This is the obvious one and it is a good test of the transcription. It is not a test of the mathematics at all: if the kernel is wrong, agreement is the wrong answer. Every defect below in a published source would have passed this check with a residual at machine precision.

Compare a solve against the solution. This is what the solutions are for, and it is how you find solver bugs. It cannot find a bug in the solution, because a wrong exact answer and a wrong solver produce the same symptom: a number that does not go to zero.

Differentiate the solution and substitute it into the equation it claims to solve. This consults nothing. If u , p , σ , η and f are what the solution says they are, then

$$\nabla \cdot \sigma + f = 0 \quad \text{and} \quad \nabla \cdot u = 0 \quad \text{and} \quad \sigma + pI = 2\eta\dot{\epsilon} \quad (1)$$

hold pointwise, and there is nowhere for an error to hide. A solution that faithfully reproduces a defective source fails this check, which is the entire reason for preferring it.

We call these the oracle-free gates. They are cheap — symbolic differentiation of expressions that already exist, sampled at a handful of points — and they run for every solution in the family on every commit.

2. ONE CONVENTION, AND IT BELONGS TO THE SOLVER

Before any of that means anything, the family has to agree on what the symbols mean. Much of the benchmark literature writes the momentum balance with the body force on the other side, or reports pressure positive in tension. A suite that silently adopted a paper’s convention would report a solver bug that was really a sign disagreement.

Underworld3’s own Stokes solver fixes the conventions, and the suite exists to validate that solver, so the solver wins wherever a source disagrees:

quantity	convention
total stress	$\sigma = \tau - p\mathbf{I}$
pressure	positive in compression
momentum balance	$\nabla \cdot \sigma + \mathbf{f} = 0$
strain rate	$\dot{\epsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T)$
boundary normal traction	σ_{nn} along the domain’s outward normal

The published sources do not agree with each other on all of this, and the disagreement is absorbed at exactly one declared boundary. Each solution carries a `stress_is_deviatoric` flag, honoured only inside the base class’s `set_fields` method, which builds the exposed stress. Four of the thirteen kernels publish the deviator τ and the rest publish the total σ ; downstream of `set_fields` there is one convention.

Two things *are* uniform across every source we vendored, and neither is stated in most of the files. Both had to be measured, by finite-differencing each kernel’s own published stress against its own published body force: the momentum sign, $\nabla \cdot \sigma + \mathbf{f} = 0$ with $\mathbf{f} = -\rho\hat{\mathbf{z}}$ under unit gravity; and the pressure sign, positive in compression.

3. THE MEASUREMENT, AND THE COLUMN THAT MAKES IT MEAN SOMETHING

Every registered symbolic Stokes solution, on a coarse box, sampled at the solution’s own sample points. All quantities are relative, normalised by the largest term being cancelled. The generating script is [examples/convention_audit.py](#).

solution	source stress	$\text{tr } \sigma + dp$	$\sigma + p\mathbf{I} - 2\eta\dot{\epsilon}$	momentum + $\mathbf{f}$	momentum − $\mathbf{f}$	$\mathbf{u} \cdot \nabla$	$\dot{\epsilon}$ vs $\nabla \mathbf{u}$
EllipticalIn- clusion	total	1.1e-15	0	3.6e-15	3.6e-15 †	1.7e-16	0
SolA	total	0	0	6.0e-17	2.0	0	0
SolB	total	2.1e-16	0	8.7e-15	2.0	4.8e-14	1.7e-14

solution	source stress	$\text{tr } \sigma + d p$	$\sigma + p \mathbf{I} - 2\eta \dot{\epsilon}$	momentum + $\mathbf{f}$	momentum - $\mathbf{f}$	$\mathbf{u} \cdot \nabla$	$\dot{\epsilon}$ vs $\nabla \mathbf{u}$
SolC	total	0	0	4.1e-16	1.8	2.8e-17	5.9e-16
SolCx	total	3.3e-16	1.0e-16	2.3e-16	1.7	3.5e-17	4.0e-15
SolDA	total	8.6e-17	1.1e-16	1.4e-15	2.0	2.4e-17	2.9e-15
SolDB2d	deviatoric	0	0	9.7e-17	0.5	8.9e-16	0
SolDB3d	deviatoric	3.9e-15	0	5.3e-17	2.0	8.9e-16	0
SolH	total	3.3e-16	0	3.8e-16	2.0	2.1e-17	2.3e-16
SolKx	total	7.4e-16	0	3.3e-16	2.0	6.9e-18	1.5e-15
SolKz	deviatoric	0	0	2.4e-16	2.0	5.2e-18	3.4e-16
SolM	deviatoric	0	0	1.7e-16	2.0	0	0
SolNL	deviatoric	0	0	4.7e-16	2.0	0	0

† EllipticalInclusion is driven entirely through its boundary and has no body force, so negating the body force is a no-op and the control cannot fire. That is a property of the problem rather than a gap in the check — there is no body-force sign to certify.

The momentum $-\mathbf{f}$ column is the load-bearing one. It re-measures the momentum residual with the body force negated, and it moves from 10^{-16} to order unity for every solution that has a body force. Without that column the table would be an assertion that small numbers are small. With it, the gate is demonstrably capable of failing, and the sign it certifies is the one the solver assembles.

One further signature is worth reading off the table. The $\sigma + p \mathbf{I} - 2\eta \dot{\epsilon}$ column contains exact zeros for most solutions and 10^{-16} for a few. The exact zeros are not better agreement — they are the solutions that published only one of stress and strain rate, so `set_fields` derived the other from precisely this identity. For those, the check is structural rather than evidential. Only SolNL, SolDB2d and SolDB3d publish both and are genuinely tested by it, and the independent counterpart for everyone else is the last column, which compares $\frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T)$ built from the *velocity* against the strain rate built from the *stress*.

4. FOUR DEFECTS

Three of these are corrections to a source and one is ours. Each was found by a gate that consults no reference, and each is recorded rather than silently applied: the vendored kernels stay verbatim, the correction lives in the transcription, and the defect stays visible to anyone auditing the provenance.

4.a. SolA's vertical normal stress is missing its viscosity

This is the one to take away from the note, and it was live in our own suite until we audited it.

The kernel computes the two normal components of the total stress on adjacent lines:

```
u3 = 2.0*kn*ss_z - pp;      /* zz total stress */
txx = -2.0*Z*kn*ss_z - pp;  /* xx total stress */
```

The xx component carries the viscosity Z and the zz component does not. The sibling kernel `solB.c` writes the same line with the Z present, and `solA`'s own deviatoric τ_{xx} carries it, so this is a defect rather than a convention.

The error is exactly $\tau_{zz}(1 - Z)/Z$, which vanishes identically at $Z = 1$. Unit viscosity is the only case the file's own driver exercises, and it was the default value of the viscosity parameter in our transcription — which is why the defect had survived being vendored, transcribed and run for years.

Measured on the transcription as it stood:

viscosity Z	momentum residual	deviator trace	$\dot{\epsilon}$ consistency
1.0	0	0	0
3.0	2.8e-1	6.7e-1	6.7e-1
0.25	6.4e-1	3.0e+0	7.5e-1

$6.7 \times 10^{-1} = |1 - 3| / 3$ and $3.0 = |1 - 0.25| / 0.25$: the predicted $(1 - Z)/Z$, exactly. `SolB` is clean at every value tested, which is the control.

Three independent gates fire, and they are independent in a way worth spelling out. The deviator is no longer traceless, which is a statement about incompressibility and has nothing to do with the body force. The strain rate no longer matches the velocity gradient. And the momentum balance fails. All three are silent at the default.

We restore the missing factor on the term that lost it rather than editing the vendored source, and we declare the correction per solution. There is an easier repair available — tracelessness gives σ_{zz} directly from the kernel's correct xx component — and we rejected it, because it would make tracelessness true by construction and so retire one of the three gates that caught the defect. The test asserts both halves: that `solA`'s published deviator is *not* traceless at $Z = 3$, and that `solB`'s is.

Anyone using this kernel at a viscosity other than 1 has a wrong σ_{zz} and no reason to suspect it.

4.b. *SolM's published stress uses the wrong viscosity*

The kernel declares its viscosity as $(1 + \cos(r\pi x))\eta_0 + 1$ and then computes its stress as $2(\eta - 1)\dot{\epsilon}$. The constant part of the viscosity is missing from the stress.

quantity	value
momentum residual, published stress	2.1e-1
momentum residual, stress from the kernel's own $\dot{\epsilon}$	1.7e-16
published τ against $2\eta\dot{\epsilon}$	3.3e-1
published τ against $2(\eta - 1)\dot{\epsilon}$	0

The last row is the argument. A transcription slip would leave a residue; an exact zero against $2(\eta - 1)\dot{\epsilon}$ says the kernel computed a self-consistent stress for the wrong viscosity, which is a defect in the source. Everything else `SolM` publishes — velocity, pressure, strain rate, viscosity, body force — is mutually consistent, so the transcription passes the strain rate to `set_fields` and lets the stress be derived.

This is the clearest case for a check that consults no reference. Comparing our SolM against its own kernel reproduces the error faithfully and reports agreement.

4.c. SolC publishes a density where its siblings publish a force

Most kernels in the family negate internally: they compute ρ and hand back $+\sigma \sin \cos$ as the force. SolC accumulates the density itself, so the transcription has to negate.

Measured: as summed, the momentum residual is 1.8; negated, 1.6e-16.

Incompressibility and the free-slip boundary conditions hold either way. This sign is invisible to everything except the momentum balance, which is the argument for having that gate at all.

4.d. The elliptical inclusion ignored its matrix viscosity

This one is ours. With a matrix viscosity of 3, the momentum residual is 6.3e-1 while the deviator trace and the strain-rate consistency stay at machine precision. That combination localises the fault immediately: the velocity, stress and strain rate are mutually consistent, but they are not consistent with the momentum balance.

The Muskhelishvili potentials are normalised to unit matrix viscosity. Under $\eta \rightarrow \lambda\eta$ at fixed boundary velocity, Stokes flow leaves the velocity and strain rate alone and scales the stress *and the pressure* by λ . The construction scaled the viscosity, and so the viscous part of $\sigma = 2\eta\dot{\epsilon} - p\mathbf{I}$, but left the pressure at its unit-viscosity value. The two parts of the stress were then in different units, which no gate looking at only one of them can see. Scaling the pressure takes the residual to 3.9e-15.

Worth recording why this one escaped for as long as it did: the elliptical inclusion is the only solution in the family that assigns its stress, pressure and strain rate directly instead of going through `set_fields`, which is the one place the stress–pressure relationship is applied.

5. A TRANSCRIPTION CAN BE RIGHT ABOUT THE SOURCE AND WRONG ABOUT THE ARRAY

SolKz is not an erratum — the kernel is correct — but it is the sharpest trap in the family for anyone transcribing afresh.

`solKz.c` computes a deviator, converts it to the total stress, and says so:

```
sum5 += u5*cos(n*M_PI*x); /* pressure */
u6    -= u5;               /* get total stress */
```

The array the function returns really is the total stress. But our transcriber reads the per-mode straight-line block and stops at the first accumulation, because the series solutions sum over modes with `+=` and the summation happens in SymPy rather than in C. The `sum5 +=` line precedes the `u6 -= u5` line, so what we capture is the value *before* the conversion — the deviator. The `stress_is_deviatoric` declaration on SolKz is correct, and it describes the transcription’s cut point rather than the kernel’s output.

Measured on the transcribed fields: read as the deviator, the momentum residual is 1.7e-16; read as the total, 6.0e-1. The deviator we capture is exactly traceless, which is the independent confirmation.

Two signatures settle this kind of question cheaply on any new kernel. A deviator is traceless, so its normal components are exact negatives of each other. And $\tau = 2\eta\dot{\epsilon}$, where the strain rate is a *different output of the*

same kernel. On SolKz the shear component agreed with $2\eta\dot{\epsilon}$ to machine precision while the normal components agreed with nothing, which located the problem in one step.

The related trap is component order. Several kernels in this family label the vertical velocity `u1` and the horizontal `u2`, the opposite of what a Cartesian (u_1, u_2) invites, and `solKx.c` orders its stress components `[xx, xz, zz]` where every sibling uses `[xx, zz, xz]`. Neither is documented in the file that does it. A swap is caught by the momentum residual only because the two components have different functional forms; in a symmetric problem it would be invisible.

6. WHAT WE WOULD TELL SOMEONE BUILDING THE SAME THING

Write the check so it cannot consult the answer. Every defect above was found by differentiating the solution and substituting it into the equation. None would have been found by comparing against the source, and the two published defects would have been actively concealed by it.

Give every gate a negative control, and run it. Not as an argument that the gate is sound, but as a measurement in the same table as the result. The body-force flip costs one extra evaluation and converts a column of small numbers into evidence.

Test away from the defaults. SolA's defect is identically zero at the one viscosity its driver exercises. A parameter sweep is now part of the suite, with a table every registered solution must appear in, precisely because the defaults are where defects go to hide.

Watch for a gate that is true by construction. If the framework derived the strain rate from the stress, then checking the stress against the strain rate checks the framework's arithmetic and nothing else. It is worth knowing which of your checks are structural, and saying so, rather than counting them all as evidence.

We had a small version of the same mistake while writing this up. The first guard that decided which solutions publish both quantities did it by walking the class hierarchy for a call to `set_fields` and reading its parameter names. It reported the elliptical inclusion as publishing both, because that solution never calls `set_fields` at all and the walk fell through to the base class and matched the parameter names in *its* signature. The check looked like it was inspecting the solution and was inspecting the thing doing the inspecting. `set_fields` now records what it was handed.

7. USING THEM

The solutions are constructed on a mesh and expose SymPy expressions, so they compose with the rest of Underworld3 directly:

```
import underworld3 as uw

mesh = uw.meshing.StructuredQuadBox(elementRes=(32, 32))
solution = uw.analytic.SolCx(mesh, eta_B=1.0e6)

stokes = uw.systems.Stokes(mesh)
stokes.constitutive_model = uw.constitutive_models.ViscousFlowModel
stokes.constitutive_model.Parameters.shear_viscosity_0 = solution.fn_viscosity
stokes.bodyforce = solution.fn_bodyforce

solution.apply_boundary_conditions(stokes)
```

```
stokes.solve()  
  
print(solution.error("velocity", stokes.u))
```

Each solution states its own boundary conditions, because they are part of the problem it answers: free slip on the walls for the Velic family, the exact velocity for the manufactured solutions, and both radii of the annulus for the cylindrical case. Every enclosed solution removes the pressure nullspace explicitly. Leaving that out is a failure mode these solutions exist to catch — a direct solve on a singular saddle returns a quiet, wrong answer with an arbitrary pressure offset, and only an exact answer exposes it.

The full family, the transcription machinery and the validation gates are in `src/underworld3/analytic/`, and the developer documentation for the subsystem covers adding a solution of your own.